Linked List Management System

Data Structures and Algorithms - Mini Project Report

Submitted by	Shahzaan Shabir Khan
System ID	2025278040
Class	CSD
Batch	Shikhar Batch 2029 - Batch 2
Language	Java

1. Objective

The objective of this project is to implement and operate singly and doubly linked lists through a menu-driven Java console application. The project demonstrates node creation, reference management, traversal, insertion, deletion, and common linked-list algorithms without using a built-in list collection as the underlying data structure.

2. Project Structure

File	Purpose
Main.java	Provides the main menu, submenus, and integer input validation.
SinglyLinkedList.java	Implements a custom node containing data and a next reference.
DoublyLinkedList.java	Implements a custom node containing data, prev, and next references.
LinkedListTests.java	Runs dependency-free regression tests for operations and edge cases.
sample-output/output.txt	Shows representative output for both types of linked lists.

3. Design Overview

The singly linked list maintains a head reference. Each node stores an integer value and a reference to the next node. Traversal proceeds from head to NULL. The doubly linked list maintains both head and tail references. Each node stores prev and next references, which permits traversal in either direction and makes insertion or deletion at the end efficient when the tail is known.

4. Implemented Operations

- Insertion at the beginning, end, a one-based position, before a value, and after a value.
- Deletion from the beginning, end, a one-based position, and by the first matching value.
- Search, update, display, count, minimum, maximum, and middle-element lookup.
- Reverse, ascending bubble sort, and removal of duplicate values.
- Forward and backward display for the doubly linked list.

5. Complexity Summary

Operation	Typical Complexity
Insert or delete at beginning	O(1)
Insert at end	O(n) for singly list; O(1) for doubly list using tail
Search, display, count, min/max, middle	O(n)
Reverse	O(n) time and O(1) extra space
Bubble sort and simple duplicate removal	O(n^2)

6. Testing and Edge Cases

The included test program exercises both implementations and checks the expected list order after operations that modify links. The tested cases include:

- Empty-list display, deletion, sorting, and reversing.
- One-based insertion at the first and last valid positions, plus invalid positions.
- Insert-before and insert-after operations at boundary nodes.
- Missing values during search, update, and deletion.
- Duplicate values, zero, and negative values.
- Forward and backward traversal after doubly linked-list changes.
- Sorting and reversing while preserving valid links.

7. How to Run

Step	Command
Compile	javac -d out src/*.java
Run application	java -cp out Main
Run tests	javac -d out src/*.java tests/LinkedListTests.java java -cp out LinkedListTests

8. Conclusion

This project provides a working comparison of singly and doubly linked lists through a single console application. It demonstrates how node references are maintained during insertion, deletion, traversal, reversal, sorting, and duplicate removal, while handling common boundary conditions safely.